

ByteHouse: ByteDance's Cloud-Native Data Warehouse for Real-Time Multimodal Data Analytics

Yuxing Han ByteDance Shanghai, China hanyuxing@bytedance.com	Yu Lin ByteDance Hangzhou, China linyu.michael@tiktok.com	Yifeng Dong ByteDance Beijing, China dongyifeng@bytedance.com	Xuanhe Zhou SJTU Shanghai, China zhouxuanhe@sjtu.edu.cn
Xindong Peng ByteDance Beijing, China pengxindong@bytedance.com	Xinhui Tian ByteDance Beijing, China tianxinhui@tiktok.com	Zhiyuan You ByteDance Hangzhou, China youzhiyuan@bytedance.com	Yingzhong Guo ByteDance Beijing, China guoyingzhong.z@bytedance.com
Xi Chen ByteDance Shanghai, China max.chenxi@bytedance.com	Weiping Qu ByteDance Shanghai, China quweiping@bytedance.com	Tao Meng ByteDance Hangzhou, China jingpeng.mt@bytedance.com	Dayue Gao ByteDance Beijing, China gaodayue@bytedance.com
Haoyu Wang ByteDance Shanghai, China wanghaoyu.0428@tiktok.com	Liuxi Wei ByteDance Singapore, Singapore weiliuxi@bytedance.com	Huanchen Zhang Tsinghua University Beijing, China huanchen@tsinghua.edu.cn	Fan Wu Shanghai Jiao Tong Univ. Shanghai, China wu-fan@sjtu.edu.cn

Abstract

With the rapid rise of intelligent data services, modern enterprises increasingly require efficient, multimodal, and cost-effective data analytics infrastructures. However, in ByteDance's production environments, existing systems fall short due to limitations such as I/O-inefficient multimodal storage, inflexible query optimization (e.g., failing to optimize multimodal access patterns), and performance degradation caused by resource disaggregation (e.g., loss of data locality in remote storage). To address these challenges, we introduce ByteHouse (<https://bytehouse.cloud>), a cloud-native data warehouse designed for real-time multimodal data analytics. The storage layer integrates a unified table engine that provides a two-tier logical abstraction and physically consistent layout, SSD-backed cluster-scale cache (CrossCache) that supports shared caching across compute nodes, and virtual file system (NexusFS) that enable efficient local access on compute nodes. The compute layer supports analytical, batch, and incremental execution modes, with tailored optimizations for hybrid queries (e.g., runtime filtering over tiered vector indexes). The control layer coordinates global metadata and transactions, and features an effective optimizer enhanced by historical execution traces and AI-assisted plan selection. Evaluations on internal and standard workloads show that ByteHouse achieves significant efficiency improvement over existing systems.

CCS Concepts

• **Information systems** → *Online analytical processing engines*; • **Computing methodologies** → *Machine learning*.

Keywords

OLAP, Data Layout, Query Processing, Multimodal Data, Intelligent Optimization

ACM Reference Format:

Yuxing Han, Yu Lin, Yifeng Dong, Xuanhe Zhou, Xindong Peng, Xinhui Tian, Zhiyuan You, Yingzhong Guo, Xi Chen, Weiping Qu, Tao Meng, Dayue Gao, Haoyu Wang, Liuxi Wei, Huanchen Zhang, and Fan Wu. 2026. ByteHouse: ByteDance's Cloud-Native Data Warehouse for Real-Time Multimodal Data Analytics. In *Companion of the International Conference on Management of Data (SIGMOD Companion '26)*, May 31–June 05, 2026, Bengaluru, India. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3788853.3803080>

1 Introduction

Real-time multimodal data analytics is becoming increasingly important for modern enterprises (see Figure 1). Across ByteDance Cloud, a diverse set of applications, including e-commerce platforms, gaming services, financial systems, and AI-powered applications, operates over large and heterogeneous data modalities. These applications rely on analytical services such as operational dashboards, real-time log analysis, and semantic retrieval over text and image data to obtain timely and reliable insights. For instance, the code assistant Trae [22] requires millisecond-level semantic retrieval across source code, documentation, and execution traces.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGMOD Companion '26, Bengaluru, India*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2450-3/2026/05
<https://doi.org/10.1145/3788853.3803080>

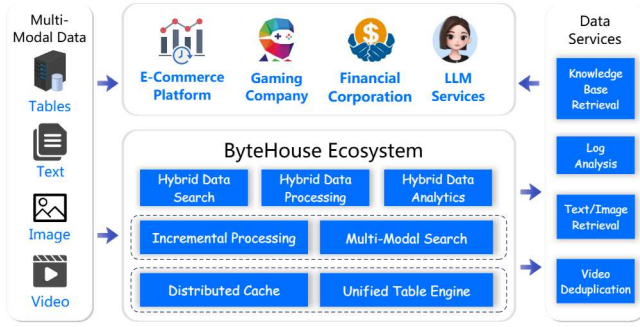


Figure 1: ByteHouse provides real-time multimodal data analytics for over 400 services across ByteDance Cloud.

Existing systems (e.g., data warehouses [1, 7, 18, 30, 37, 59, 73, 75], vector databases [47, 67]) have limitations in meeting these demands. First, many analytical engines [30, 75] offer limited computation optimization for such multimodal workloads. For instance, they lack fine-grained incremental processing at operator level (e.g., delta-aware computation) and the data level (e.g., handling newly ingested rows). Additionally, their query optimizers are typically based on static rule- or cost-driven strategies and lack adaptive mechanisms that optimize evolving query patterns (e.g., text-vector fusion, scalar-constrained vector search) based on historical executions. Existing vector databases [67] and plugin-based solutions [47] also struggle to efficiently optimize such hybrid queries. Second, many cloud-native architectures remain bottlenecked by remote object storage. Although disaggregated storage enables elasticity, it introduces significant latency for sparse or point-access workloads. Even with enhancements like vectorized execution or RDMA-backed memory sharing [37, 73], performance degrades substantially for small or random reads, making real-time responsiveness difficult to sustain. Furthermore, in many cloud-native analytical systems, file-level metadata and auxiliary indexing structures are maintained separately from base data files and stored in distinct object-store locations [18, 52]. Because data files are distributed across object-store partitions and accessed via remote metadata lookups, query processing may incur additional I/O amplification and metadata consistency overhead.

To address these challenges, we propose ByteHouse, a cloud-native analytical data warehouse built upon a shared-storage architecture, which has been refined through *hundreds of application scenarios and tens of thousands of users*. (1) The storage layer provides a unified table engine with a self-describing columnar file format (Sniffer) that natively eliminates external metadata/index dependencies. And ByteHouse provides an SSD-backed distributed cache plane (CrossCache) that can independently scale and close the performance gap of traditional compute-storage separation. Additionally, ByteHouse supports efficient local data access through an alignment-aware filesystem abstraction on each compute node. (2) The compute layer offers three execution modes (i.e., analytic pipeline, staged batch, incremental processing) to handle diverse workloads such as real-time analytical processing. Moreover, we introduce hybrid retrieval operators (e.g., RANK_FUSION) for weighted semantic-lexical scoring, scalar-predicate-aware optimizations (e.g., runtime filtering across joined tables), and a tiered vector index

design tailored to different service requirements. (3) The control layer manages global metadata, transactions, placement, and scheduling across the distributed architecture. Additionally, the query optimizer supports history-based optimization (HBO), which reuses runtime statistics from past executions to refine selectivity, cardinality, and operator-cost estimates. Building on HBO, ByteHouse incorporates AI models to learn correlations among query structures, data distributions, and runtime behavior to generalize beyond previously observed plan fragments, such as for predicate push-down selection and join side selection.

Contributions. In summary, we make the following contributions:

- (1) We propose ByteHouse, a cloud-native, shared-storage data warehouse for real-time multimodal data analytics, supporting high-throughput ingestion and ultra-low latency hybrid queries.
- (2) We design a vertically integrated storage layer with a unified table engine, self-describing file format, SSD-based chunk-level caching, and a buffer-managed virtual filesystem that reduces I/O overhead across both the storage and compute nodes.
- (3) We propose a unified execution framework combining analytic, batch, and incremental modes, with fusion-based retrieval operators and hybrid query optimization for complicated workloads.
- (4) Extensive evaluations on both public benchmarks and ByteDance’s internal workloads show that ByteHouse outperforms existing systems on both analytical queries (over 25% lower latency in ClickBench) and multimodal data processing (over 50% higher throughput in Cohere and C4).

2 ByteHouse Architecture

Figure 2 illustrates the architecture of ByteHouse, a cloud-native analytical warehouse built upon a shared-storage design that fully decouples the control, compute, and storage services.

Control Layer. This layer provides global coordination within the distributed framework. The Server handles SQL parsing, semantic analysis, hybrid plan optimization (by history execution statistics and regression models), and distributed scheduling, translating user requests into executable workflows. The Control Services govern global metadata, placement, and system-wide transaction coordination. The Catalog Manager stores versioned metadata in ByteKV, exposing snapshot-consistent schemas, partitions, and index definitions across concurrent operations. The Global Transaction Manager issues globally ordered commit timestamps to ensure serializable transactions and consistent snapshot reads. The Daemon Manager orchestrates background maintenance tasks such as compaction and merge scheduling.

Compute Layer. The compute layer hosts a pool of stateless workers operating in a hybrid execution model with three modes: (1) *Analytic Pipeline Mode (APM)* for distributed multi-stage execution with shuffle, gather, and broadcast exchanges; (2) *Staged Batch Mode (SBM)* for long-running ETL with staged retries and shuffle persistence; (3) *Incremental Processing Mode (IPM)* for delta-aware computation with lineage tracking and versioned operator maintenance. All modes share a unified optimizer and runtime to enable seamless transitions. To support multimodal retrieval and analytics, the compute layer provides a spectrum of index structures, including Min-Max, Set, Bloom filters, and vector indexes [28, 40]. ByteHouse additionally introduces hybrid retrieval operators (e.g.,

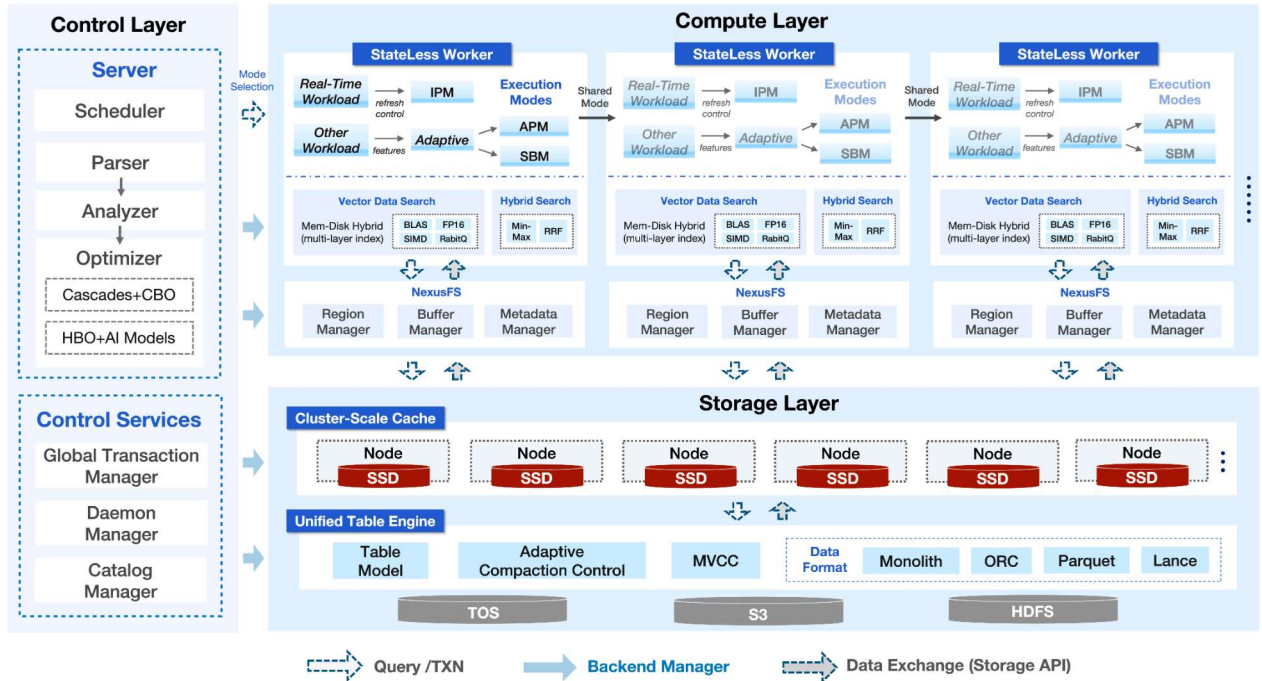


Figure 2: ByteHouse Architecture.

RANK_FUSION), scalar-predicate-aware optimizations (e.g., runtime filters pushed into vector scans), and a tiered vector indexing design tuned for online, near real-time, and cost-sensitive services.

Storage Layer. This layer provides persistent and scalable multimodal data management through three tightly integrated components. (1) The *unified table engine* offers a two-tier logical abstraction (documents–chunks) and a physically consistent layout composed of stable and delta segments. It manages ingestion through a staging–flush write path, maintains snapshot-consistent visibility via MVCC, and supports adaptive compaction and tiered point-lookup resolution essential for analytical workloads. (2) The *distributed cache plane* comprises SSD-backed cache nodes that partition data into fine-grained chunks, apply consistent hashing for balanced placement, and employ prefetching and asynchronous flushing to mitigate remote-storage latency and reduce I/O amplification. (3) The *virtual filesystem abstraction* (NexusFS) unifies access to heterogeneous storage backends, such as TOS (internal object storage), HDFS, and local SSDs, under a single logical namespace. NexusFS provides alignment-aware region management, coordinated metadata lookup, and buffer orchestration to sustain high-throughput read/write paths from compute nodes. Persistent data is stored in columnar formats such as Parquet, ORC, Lance, and the self-describing Sniffer format, enabling efficient hybrid processing across structured, textual, and vector modalities. All compute–storage data exchange is performed through Arrow-based [62] interfaces for zero-copy transfer and high-throughput communication across the disaggregated architecture.

3 ByteHouse Storage

This section presents the storage design of ByteHouse for managing large-scale multimodal data in the cloud. We first introduce the Unified Table Engine, which provides a two-tier logical abstraction and

physically consistent layout. We then detail a self-describing file format that co-locates raw data, indexes, and metadata to accelerate operations like point lookups. Next, we describe the distributed cache plane (CrossCache), which mitigates I/O bottlenecks in disaggregated architectures. Finally, we discuss NexusFS, a virtual file system that unifies local disk caching with remote CrossCache access for efficient compute-side reads.

3.1 Unified Table Engine

ByteHouse increasingly serves heterogeneous analytical workloads that span traditional OLAP queries, real-time incremental refresh, and multimodal data analysis. From the perspective of the storage engine, OLAP queries rely on large-scale scans and aggregations over immutable columnar data; incremental pipelines require efficient ingestion and low-latency access to recently updated results; and multimodal workloads often perform feature-level retrieval, such as fetching semantic vectors by primary or sort keys, where point-lookups and fine-grained access latency become critical.

Existing systems typically rely on separate engines to handle these diverse patterns, leading to duplicated metadata and inconsistent data freshness [26]. Instead, ByteHouse introduces a Unified Table Engine that combines these capabilities under a single abstraction. It provides a consistent table model, unified metadata and versioning, and transactional visibility across analytical, incremental, and multimodal workloads.

3.1.1 Table Model Design. ByteHouse introduces a table model that defines how multimodal data are logically represented and physically organized. Each table encapsulates a unified schema that can contain structured attributes, nested fields, and vector columns derived from unstructured modalities. All columns are versioned and tracked by logical descriptors that record data type, encoding,

and statistical summaries. Tables define *primary keys* and *sort keys*, which together enable efficient range pruning for analytical scans and provide low-latency point lookups for feature retrieval tasks common in multimodal workloads.

(1) Logical Table Design. On top of this table foundation, we introduce a two-tier logical abstraction for multimodal data organization. Each multimodal dataset is represented as a single table comprising a collection of *documents*, where each document denotes a logical entity such as an article, image, or video. Each document is further decomposed into semantically coherent *chunks*, which constitute the fundamental units for embedding-based retrieval and analytical operations. Every record in the table is uniquely identified by a composite key (document_id, chunk_id).

(2) Physical Table Design. Each table is physically composed of *stable segments* and *delta segments* that collectively maintain a consistent version chain. Stable segments store immutable, columnar data optimized for analytical throughput, while delta segments capture recent physical updates, inserts, and compact feature refreshes. Multi-version concurrency control (MVCC) governs the visibility of data across stable and delta segments, ensuring that analytical queries always read from a consistent snapshot even as refresh operations update the underlying data. Background merge and compaction tasks asynchronously incorporate delta segments into stable segments, maintaining high ingestion performance while progressively organizing historical data.

3.1.2 Adaptive Compaction Control. Furthermore, efficient compaction of these delta segments is essential for controlling storage costs and sustaining query performance in large-scale data warehouse storage systems [3, 26]. However, the challenge is that overly frequent compaction amplifies write I/O and inflates storage costs, especially in cloud-native deployments where each write incurs overhead from remote storage, and insufficient compaction leads to segment fragmentation and degraded scan locality over time instead. Thus, ByteHouse introduces an adaptive compaction control mechanism that modulates compaction intensity to resolve the (observed) accumulated delta segments.

Specifically, let N_Δ denote the number of active delta segments, and N^* the equilibrium level that characterizes a balanced compaction state. ByteHouse regulates compaction aggressiveness by a **bounded linear controller**, where the compaction intensity $\alpha \in [0, 1]$ is defined as:

$$\alpha = \min \left(1, \max \left(0, k \cdot \left(\frac{N_\Delta}{N^*} - 1 \right) \right) \right), \quad (1)$$

with k controlling the sensitivity to segment accumulation. The coefficient α governs the triggering frequency, merge batch size, and scheduling priority of background compaction tasks. When the number of delta segments is near the equilibrium level, the controller keeps α low, resulting in conservative compaction that avoids redundant compaction work. As N_Δ increases, α increases linearly, progressively relaxing merge thresholds and enlarging compaction batches until the controller saturates at full compaction intensity. This monotonic linear control provides smooth transitions between idle and aggressive compaction phases, preventing oscillation and maintaining steady ingestion performance.

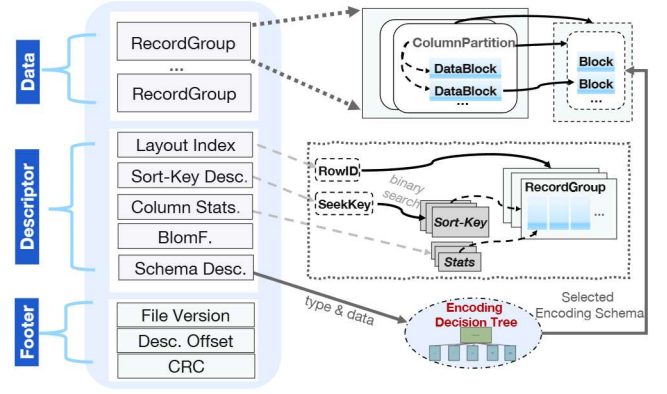


Figure 3: Sniffer File Format.

3.1.3 Tiered Storage Pipeline. To efficiently handle small and frequent row-level ingestion without sacrificing columnar efficiency, ByteHouse employs a two-stage write pipeline that decouples transient ingestion from durable columnar persistence [6, 34].

(1) Staging in Key-Value Store. Incoming updates are first written to a lightweight *staging area*, supported by a distributed key-value store at ByteDance, ByteKV. Each write is assigned a transaction identifier and recorded in a Write-ahead Log to ensure durability and atomicity. The staging area serves as a short-lived buffer that accumulates recent changes in row-oriented format.

(2) Flush to Columnar Storage. Once the data size or retention time exceeds predefined thresholds, ByteHouse flushes the buffered data to columnar storage, reorganizing it into compressed column segments. During this flush process, schema evolution and version visibility are preserved, allowing newly persisted data to integrate seamlessly with existing analytical files.

To support low-latency point access, ByteHouse resolves lookups across both staging and columnar layers under snapshot isolation. The engine first probes the transactionally consistent staging store for the most recent visible version of a tuple and falls back to columnar data parts when necessary, leveraging part-level pruning metadata to minimize I/O. This tiered resolution follows the general separation of write- and read-optimized storage and ensures consistent visibility without requiring full-part scans.

3.2 Self-Describing File Format

Next we introduce the file format in ByteHouse. In traditional column stores, column data and auxiliary indexes are maintained as separate structures. While this organization is effective for large sequential scans, it forces point and small-range queries to open additional files, issue seeks, and perform metadata lookups, leading to substantial I/O amplification and metadata overhead in lookup-intensive or hybrid analytical workloads. Furthermore, the common practice of representing Array columns as flattened offset-value pairs lacks efficient support for modern multimodal data, such as high-dimensional embedding vectors.

To overcome these limitations, ByteHouse adopts a self-describing file format (Sniffer) that co-locates data blocks with their file-level indexes and schema metadata within a single cohesive layout. This way, Sniffer eliminates external index and metadata files and enables self-contained data access with minimal I/O overhead.

3.2.1 File Organization. Sniffer is composed of Data, Descriptor, Footer regions in a hierarchical structure (see Figure 3).

(1) Data Region stores compressed columnar data arranged in three levels: (i) A *RecordGroup* corresponds to a batch of rows that share the same schema and partitioning key, analogous to a row group in Parquet or a stripe in ORC. (ii) Each record group contains multiple *ColumnPartitions*, each representing a contiguous subset of a column's values within the group. (iii) Every column partition consists of one or more *DataBlocks*, the smallest physical units that store compressed, type-specific blocks of values.

(2) Descriptor Region maintains all structural metadata required to interpret the Data Region efficiently: (i) The *Layout Index* maps *RecordGroups* and *ColumnPartitions* to their physical *DataBlocks* and file offsets; (ii) The *Sort-Key Descriptor* records ordering information for binary search and seek-key resolution; (iii) The *Column Statistics* store block-level summaries such as min/max and null counts for predicate pruning; (iv) The *Bloom Filter* provides coarse-grained filtering for selective queries; (v) The *Schema Descriptor* defines the logical schema, data types, and encoding metadata (e.g., compression codec) used in the file. Note, all metadata are file-scoped, which can be co-located with the storage layer within the file or maintained in a separate search acceleration layer [48].

(3) Footer Region serves as the global directory and integrity anchor of the file [66, 74]. It records the physical descriptor offsets, the file version, and the Cyclic Redundancy Check (CRC) values across the data and descriptor regions. This design allows the reader to locate and validate all descriptors with a single footer access, enabling complete reconstruction of the file layout without relying on external catalogs. Besides, the footer preserves version metadata to ensure backward compatibility and supports hierarchical integrity verification across regions.

Built upon this file format, we can accelerate fine-grained access to compressed columnar data by minimizing random I/O. At query time, the Layout Index, Sort-Key Descriptor, and Column Statistics collectively provide precise navigation over the hierarchical storage structure, allowing the engine to identify and load only the data blocks relevant to a given predicate, including key-based lookups. For point lookups, the engine first identifies the target *RecordGroup* via the Sort-Key Descriptor, then consults the Layout Index to fetch the precise *DataBlock* offsets. Because both descriptors are memory-resident, each lookup typically requires only one metadata seek and one block read, achieving *microsecond-level* latency even under high analytical concurrency.

3.2.2 Column Encoding Schemes. Additionally, the column encoding methods are critical for the lookup efficiency (e.g., I/O, decoding overhead). Thus, Sniffer dynamically encodes each column according to its data type and statistical distribution [33]. During write time, ByteHouse samples each column and selects the encoding scheme that minimizes both storage footprint and decoding cost. Sniffer integrates several state-of-the-art encoding schemes for different data characteristics: Frame-of-Reference (FOR) combined with Bitpacking [36] with narrow value ranges; Run-Length Encoding (RLE) for low-cardinality columns with frequent repetitions;

Dictionary Encoding for categorical strings and enumerations, reducing both size and comparison overhead during predicate evaluation; Finite-State Symbol Table (FSST) [10] for general text or high-entropy string fields; and Adaptive Losses Floating Point (ALP) [2] for floating-point columns, where precision can be adaptively reduced without compromising analytical accuracy.

3.2.3 Format Optimization for Vector Data. In general column stores [18, 30, 52], vectors are stored as flattened Array columns in the form of two companion buffers, i.e., an offset array indicating element boundaries and a value array storing all elements contiguously. While compact, this organization introduces several inefficiencies when applied to nested or vectorized data. First, pooling all vector elements into a single value buffer prevents the system from maintaining per-vector statistics or applying compression schemes that operate at vector-block granularity. Second, sparse or variable-length vectors must be represented using placeholder offsets or filler values, resulting in unnecessary storage space waste.

To address this, Sniffer adopts the Length-and-Presence (L&P) representation [44] instead of the repetition/definition encoding used by Parquet [43]. L&P attaches lightweight length and presence metadata to each vector, which enables Sniffer to isolate every embedding as an independent physical unit. This structure enables the system to maintain vector-level statistics, such as value ranges, norms, and nullness, within the Descriptor Region. Besides, it naturally captures sparse or variable-length embeddings without requiring padding or auxiliary offset structures. Therefore, storage cost scales with actual content rather than with the declared dimensionality. Since each vector is stored as a contiguous slice, block-oriented encodings such as FOR or ALP can be applied independently to each vector block. This improves locality and enables SIMD-efficient decoding for dense numeric vectors.

3.3 Cluster-Scale Caching on Storage Side

In cloud-native architectures, decoupling elastic compute from shared storage exposes a substantial performance gap when analytical workloads operate on cold or partially cached data. As a result, query execution often becomes I/O-bound, with remote storage accesses introducing high latency. These factors amplify tail stalls and frequently dominate end-to-end analytical response time. Production traces from ByteHouse deployments show that small-block I/O can vary by more than an order of magnitude across percentiles, with over 70% of execution time spent on cold reads in I/O-intensive workloads. As the cluster scales, read amplification and tail latency increasingly constrain analytical performance.

Distributed Cache Design. We design CrossCache, a distributed caching plane that mitigates remote-I/O bottlenecks by serving hot data from nearby nodes. CrossCache is composed of two main parts: (1) Cache Coordinators (CCs) that manage the global namespace and metadata, and (2) Cache Nodes (CNs) that provide local SSD-based caching and interact directly with different storage backends. Each cached file is divided into fixed-size blocks (12 MB by default), with block metadata and placement information maintained by the CNs. CNs handle both read and write operations. For read paths, when a file is first accessed, the CN checks whether its metadata exists in the cache namespace; if absent, it retrieves the metadata from the backend, partitions the file into blocks, and registers them

for subsequent access. For write paths, CNs buffer new data locally and coordinate block uploads once the buffers are filled. Small blocks are merged into unified files for distributed file systems or concatenated into larger objects for object stores. CNs periodically report block mappings and metadata to the CCs, which consolidate these updates to maintain global consistency across the cluster.

Fine-grained Chunk Caching. To further mitigate I/O amplification and improve cache effectiveness under highly fragmented analytical access patterns, CrossCache adopts a fine-grained chunk caching strategy. Each cached data block is further divided into 4 MB chunks, which are indexed in memory at each CN. Contiguous chunks are appended sequentially to the SSD-resident block file, whereas non-contiguous chunks are temporarily buffered until they can be coalesced into a continuous range. To address the limited write concurrency of storage backends such as HDFS (where each file admits only a single writer), CrossCache employs a *parallel flushing* mechanism. Multiple CNs upload their completed block files concurrently as temporary objects, fully utilizing available network and I/O bandwidth. A lightweight concat operation later merges these objects into a single backend file. This design substantially improves write throughput and reduces persistence latency under data-intensive workloads.

3.4 File System Abstraction on Compute Side

While CrossCache enables shared caching across compute nodes, each compute worker still requires an efficient local access path to interface with it. In this context, a data fragment refers to a logical sub-range of a file requested by the execution engine.

In many existing local caching mechanisms, these fragments are handled independently at arbitrary byte offsets [9]. When such requests do not align with the underlying storage block boundaries (e.g., SSD page size or object-store block size), the system must perform unaligned I/O, meaning that the requested byte range spans multiple physical blocks or requires partial-block reads. Frequent unaligned small I/Os degrade throughput, waste SSD and network bandwidth, and undermine the effectiveness of prefetching, especially in scan-heavy analytical workloads.

NexusFS bridges this gap by introducing an alignment-aware file-system abstraction that unifies local disk caching and remote CrossCache access. It enforces end-to-end alignment across the remote data layout, on-disk cache regions, and in-memory buffers, ensuring that data is stored and accessed in coalesced and sequential units across all layers. Internally, NexusFS consists of three tightly integrated components: a region manager, a buffer manager, and a metadata manager. The *region manager* partitions local disk space into fixed-size regions, typically 1 MB each, which are further divided into data segments that serve as the basic units of caching and I/O scheduling. It maintains a global index mapping logical file offsets to cached segment locations and reclaims space through a FIFO-based eviction policy [72]. The *buffer manager* governs in-memory caching through a fixed-size pool of segment-aligned buffers, applying a second-chance replacement policy to balance reuse and concurrency. When a segment is retained for query processing, it can be directly exposed to the execution pipeline, enabling *zero-copy* reads. The *metadata manager* maintains a hierarchical namespace for all files accessed through NexusFS, using a two-level hash hierarchy to track cached segments efficiently [76].

This structure supports constant-time lookups while minimizing memory overhead by selectively serializing inactive entries.

4 Execution Engine

ByteHouse provides three execution modes: the Analytical Pipeline Mode (APM) for low-latency interactive analytics, the Staged Batch Mode (SBM) for high-throughput offline pipelines, and the Incremental Processing Mode (IPM) for continuously ingesting new data. APM and SBM are coordinated by a mode-adaptive execution framework that uses an AI-assisted selector to extract runtime features, predict resource demand, and route each query to the more efficient mode. For IPM, ByteHouse incorporates a rule-based refresh controller that adjusts the refresh interval using a lightweight stabilization policy to balance freshness and maintenance cost.

4.1 Execution Modes

This subsection presents the three execution modes of ByteHouse.

4.1.1 Analytic Pipeline Mode. To accelerate analytical workloads, ByteHouse supports an Analytic Pipeline Mode (APM) that optimizes query planning and resource allocation for low-latency analytical processing. At its core, APM adopts a vectorized, pipeline-parallel execution model that processes columnar data in CPU-efficient batches, achieving high instruction throughput with minimal scheduling overhead. APM incorporates several operator-level optimizations to accelerate core analytical primitives.

For aggregations, it employs an *adaptive* mechanism that samples early input to estimate grouping-key cardinality and the reduction ratio, dynamically choosing between partial aggregation or direct shuffling followed by merge aggregation to avoid unnecessary local grouping and redundant merge computation. For joins, APM applies **runtime filters** generated by build-side operators and pushes them down to probe-side scans, enabling early elimination of non-matching join keys and significantly reducing both probe-side data volume and the irregular, non-sequential access patterns that typically arise during join probing.

To maintain stable pipeline execution under high concurrency, APM integrates a **credit-based flow control** scheme in which each downstream operator grants a bounded number of processing credits to its upstream producers. This regulates data production and buffer occupancy, preventing excessive queuing and sustaining steady pipeline throughput. APM also supports an **ordered consumption** mechanism that preserves the order of intermediate results when the upstream operator already produces ordered output. The consumer incrementally merges incoming ordered segments as they arrive, avoiding a full materialization-and-sort step and reducing overall query latency.

4.1.2 Staged Batch Mode. While APM targets interactive and low-latency analytical workloads, many production scenarios, such as large-scale ETL jobs and LLM-oriented data normalization pipelines, require long-running and throughput-oriented execution. To support these workloads, ByteHouse provides a Staged Batch Mode (SBM), a stage-based execution framework built for stability, recoverability, and sustained throughput. SBM divides a query plan into a sequence of stages separated by *exchange* boundaries. Each stage is decomposed into parallel tasks that operate on disjoint

data partitions, and a stage begins execution only after all of its upstream dependencies have completed. Within a stage, tasks can **materialize intermediate results** to temporary storage, allowing downstream tasks to retrieve their inputs from local or remote spill files based on data locality. These materialized outputs act as lightweight checkpoints and enable **task-level retries**, where failed tasks can be relaunched without restarting the entire stage, significantly reducing recovery time and operational overhead. SBM further supports **elastic parallelism**: input partitions assigned to a compute worker can be processed in multiple batches, allowing the system to flexibly increase task parallelism while keeping each task's memory footprint small.

4.1.3 Incremental Processing Mode. To support near real-time analytical workloads, IPM maintains continuously updated query results as base tables evolve. Unlike APM and SBM, which execute queries by recomputing their inputs in a full scan or a stage-based batch pipeline, IPM maintains operator state across refresh cycles and evaluates only the data deltas produced since the previous materialization point. This incremental model significantly reduces computation cost and end-to-end latency while preserving transactional consistency [4, 11, 58, 68]. Realizing IPM requires coordinated changes across the query processing stack.

SQL Semantics and Triggering: IPM remains fully compatible with standard syntax and extends DML statements with *refresh interval* annotations that specify the desired maintenance frequency. Shorter intervals provide fresher results at higher compute cost, whereas longer intervals reduce system load while maintaining consistency guarantees.

Row-Level Lineage and Delta Model: IPM introduces *row-level lineage tracking* to support deterministic and fine-grained incremental maintenance. Unlike simple modification flags that merely indicate whether a row has changed, ByteHouse associates each tuple with two immutable metadata fields: i) *tuple_key*, which uniquely identifies the logical identity of a row across versions; ii) *update_seq*, a monotonically increasing sequence number that records the version order of updates for that logical row.

During incremental execution, operators consume streams of delta tuples annotated with this lineage metadata. The lineage information is systematically propagated along the operator pipeline to preserve traceability. For example, in an inner join, the output tuple inherits a composite lineage derived from the participating base tuples. When a base tuple is updated or deleted, the engine uses its *tuple_key* to locate and retract previously materialized join results associated with earlier *update_seq* values. This ensures correctness under incremental recomputation without requiring full re-scans of base tables.

Operator-Level Delta Execution: All incremental operators consume and emit delta records of the form $\langle \text{tuple_key}, \text{update_seq}, \text{op} \rangle$, where $\text{op} \in \{\text{insert}, \text{delete}\}$. A logical update generates a delete delta for the previous version, followed by an insert delta for the new version. Upon receiving a delete delta, an operator locates previously materialized state entries (e.g., stored in state tables) derived from the given *tuple_key*, removes their contribution, and propagates a corresponding delete delta downstream. This uniform delta protocol ensures compositional retraction across operators.

For aggregation, IPM maintains a state table keyed by grouping attributes, where each entry materializes the current aggregate state, including partial aggregates and any derived results. Incoming deltas trigger state lookups and either apply new values or retract obsolete ones, while groups whose aggregate values (e.g., COUNT) drop to zero are eliminated through lightweight deletions. Aggregate maintenance is specialized according to algebraic properties. For natively retractable aggregates, such as COUNT, SUM, and AVG, IPM adopts fully incremental implementations. Insert deltas are accumulated into the maintained state, whereas delete deltas are subtracted from their prior contributions. In contrast, aggregates that do not efficiently support retraction under arbitrary update streams, such as MIN, MAX, are handled using a fallback strategy. IPM retains the original per-group values within the operator state to enable recomputation confined to the affected group. When a delete delta invalidates the current aggregate result, for example, by removing the current minimum, the operator recomputes the aggregate over the affected group. This design preserves correctness while confining recomputation to impacted groups, trading additional memory overhead for bounded recomputation cost.

For inner joins, the optimizer rewrites each join into three delta subqueries that combine the left delta with the right base, the right delta with the left base, and both deltas together. These subqueries execute in parallel over versioned inputs managed by the GTM to ensure snapshot consistency. Their outputs are then unified through a lineage-based reconciliation step that merges overlapping tuples by *tuple_key* and modification sequence.

For left or right outer joins, IPM augments the plan with correction terms that maintain the correct semantics of null extension. The system tracks whether each row currently has at least one matching partner and emits updates whenever this match status changes. When an unmatched row gains a match, its null extended output is withdrawn, and the matched rows are emitted; when the final match disappears, the matched rows are retracted, and a single null extended row is produced.

4.2 Adaptive Execution and Refresh Control

To balance responsiveness, throughput, and data freshness, ByteHouse employs a learning-based mode selector that adapts query execution to workload characteristics, and a rule-based refresh controller that schedules incremental maintenance for IPM.

4.2.1 Adaptive Mode Selection Between APM/SBM. The learning-based mode selection framework performs predictive mode selection using a lightweight regression model. Upon query submission, a feature extractor parses the SQL statement and derives a composite query feature vector from the query plan. The vector consists of three parts: 1) query-level features, such as query length and number of views, 2) access-pattern features, including a one-hot encoding of referenced tables, and 3) plan-structural features obtained by bottom-up traversal of plan nodes, where each node is represented by an M -dimensional structural vector. This feature vector is fed into a regression model trained from historical executions. The model jointly predicts query latency, CPU usage, and memory demands, which are then mapped to mode classes (i.e., APM or SBM) using *multi-dimensional percentile* thresholds derived from the cluster's recent workload statistics. These thresholds are

continuously recalibrated to capture runtime variations in resource contention and query concurrency, ensuring balanced mode selection between responsiveness and throughput. The prediction model is periodically retrained with recent execution statistics to maintain accurate and adaptive resource estimation.

4.2.2 Rule-based Refresh Control. Currently, IPM is explicitly triggered by users when continuous or periodic data maintenance is required. Automatically determining an appropriate refresh interval is crucial for incremental processing, as manually configuring a fixed refresh interval can lead to either excessive recomputation or stale data under varying workloads. To address this, ByteHouse employs a window-based stabilization policy that smooths transient fluctuations in maintenance cost and ensures convergence toward a steady refresh frequency. Instead of relying solely on the most recent observation, the policy maintains a sliding window of execution times from recent refresh rounds, denoted by $T_1, T_2, \dots, T_N$. It estimates the average incremental maintenance cost as:

$$T_{avg} = \frac{1}{N} \sum_{i=1}^N T_i, \quad (2)$$

where N is typically small (e.g., 3-5) to balance stability and responsiveness. The next refresh interval is computed as:

$$\Delta t = \min(\max(k \times T_{last}, \Delta t_{min}), \Delta t_{max}(U)), \quad (3)$$

which explicitly restricts the interval to lie within a lower bound Δt_{min} and an upper bound $\Delta t_{max}(U)$. The scaling factor k prevents overly frequent maintenance, while Δt_{min} protects the system from thrashing. The upper bound $\Delta t_{max}(U)$ adapts to real-time cluster utilization U : ByteHouse enlarges it under high CPU or I/O load to reduce contention, and contracts it under low load to improve freshness. A simple linear model is used in practice:

$$\Delta t_{max}(U) = \Delta t_{base} \times (1 + \alpha U), \quad (4)$$

where Δt_{base} denotes the nominal upper bound under idle conditions, and α controls the sensitivity to load variation. By averaging recent refresh costs and adapting to real-time cluster utilization, this policy absorbs transient spikes, prevents runaway growth of Δt , and converges to a stable refresh frequency that balances data freshness with overall system efficiency.

5 Query Optimization

This section describes ByteHouse's query optimization techniques. Section 5.1 describes the optimizer architecture and its Cascades-based rewrite and enumeration framework, while Section 5.2 introduces AI-driven methods for predicate pushdown selection and join side selection.

5.1 Cascades Optimizer

ByteHouse's query optimizer builds upon the Cascades framework [24, 25, 57] and extends it with a unified cost-based model (CBO) that reasons about partitioning, sorting, and grouping properties during query optimization [77]. Each operator is annotated with structural metadata describing data distribution and ordering, allowing the optimizer to derive and enforce these properties consistently throughout plan enumeration. To efficiently explore the

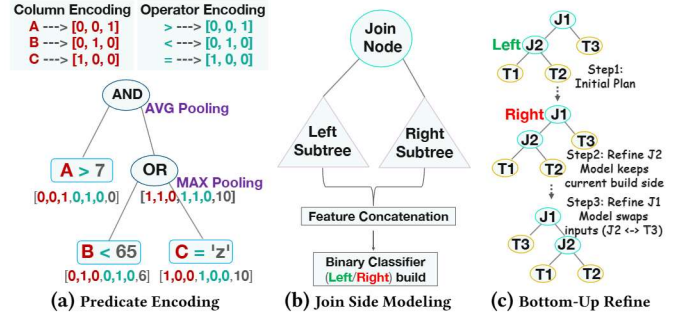


Figure 4: Intelligent Optimization by History Executions.

search space of bushy join trees, ByteHouse adopts the branch partitioning top-down enumeration algorithm [20], which enumerates join combinations in constant time and eliminates the complex data structures required by earlier approaches. The optimizer further incorporates a Magic Set rewriting [53, 54] to enhance predicate selectivity propagation. By replicating selective subplans into upstream operators, this transformation enables early filtering and significantly reduces intermediate results. ByteHouse also employs a cost-based optimization for Common Table Expressions (CTEs), where the optimizer decides adaptively whether each CTE should be inlined, shared, or materialized based on its contextual reuse and query semantics [19, 56].

5.2 Intelligent Optimization

To enhance the robustness and accuracy of query optimization in dynamic analytical workloads, ByteHouse adopts a history-based optimization (HBO) [21, 55] that leverages runtime statistics collected from past query executions to guide future cost-based decisions. Each plan fragment is transformed into a canonical representation and assigned a hash value that serves as a lookup key in a lightweight history store. During query compilation, the optimizer performs hash-based matching to retrieve previously recorded statistics, such as predicate selectivities and join cardinalities, and incorporates them into the cost estimation. While HBO improves the robustness and accuracy of cost estimation by reusing statistics from past executions, it remains fundamentally limited to plan fragments observed before and cannot fully accommodate new query patterns or evolving data distributions.

To overcome this limitation, ByteHouse incorporates regression models to learn correlations among query structures, data characteristics, and runtime behavior [41, 42, 49, 60, 64, 78, 79]. By capturing these learned relationships, ByteHouse can generalize beyond hash-based plan reuse and provide more accurate cost estimates for previously unseen workloads. Beyond ByteCard [27], which leverages probabilistic graph models [32] to refine cardinality estimation for base tables (e.g., filter selectivity and join size), we further deploy learning-based optimizations targeting Predicate Pushdown Selection (PPS) and Join Side Selection (JSS). Both optimizations have been validated in production deployments.

1) Predicate Pushdown Selection. While predicate pushdown [29, 71] is commonly regarded as a straightforward optimization, its impact in ByteHouse is more nuanced due to disaggregated storage and multimodal execution. In particular, aggressively pushing down predicates whose evaluation cost is high relative to their

selectivity benefit (e.g., vector similarity conditions or UDF-based filters) may increase remote I/O and processing overhead while precluding more effective pruning opportunities later in the plan. Therefore, predicate pushdown in ByteHouse requires cost-aware selection rather than indiscriminate application.

We formulate this decision process as a supervised regression task for estimating predicate-specific I/O costs. In the offline stage, we collect training samples of (predicate, I/O cost) pairs, and train a regression model that learns the mapping from predicate features to observed scan costs. At runtime, ByteHouse leverages the trained model to evaluate candidate predicates and select the most cost-effective ones for pushdown.

To construct candidate predicates, ByteHouse decomposes each query’s WHERE clause into top-level conjunctive components separated by AND operators, yielding independent candidates for pushdown evaluation. Each candidate is represented as an Abstract Syntax Tree (AST) whose nodes correspond to comparison operators, column references, literal values, etc. For node encoding, we apply one-hot encodings to categorical features such as comparison operators and column identifiers, while continuous features are discretized into value-domain buckets following prior work [31, 70]. For AST encoding, conjunctions and disjunctions (AND, OR) are modeled as pooling functions [23] over their child encodings rather than as categorical tokens. This approach can preserve the logical semantics of predicate composition. Specifically, OR applies MAX pooling and AND applies AVG pooling. The final predicate embedding is obtained by a postorder traversal across the AST that recursively aggregates node representations. Figure 4a presents an example of predicate encoding for: $(A > 7) \text{ AND } (B < 65 \text{ OR } C = 'x')$. Note, the underlying relational table contains three columns (A, B, C), and the analytical workload involves three comparison operators ($>$, $<$, $=$); columns A and B share a numeric domain of $[0, 100]$, and C is a categorical attribute with 10 distinct values.

2) Join Side Selection. Join performance is highly sensitive to which input is chosen as the *build* (hashed) side versus the *probe* side, as this decision governs hash-table size, memory pressure, and downstream I/O. Traditional optimizers rely on cost models to select the build side, but their effectiveness hinges on accurate cardinality estimates and may degrade under skew or complex predicates. In ByteHouse, we introduce a learning-based side-selection mechanism that refines the initial join physical plan produced by the optimizer. As illustrated in Fig. 4b, we formulate build/probe selection for a join node as a binary classification task that predicts which child should serve as the build input. For each join node, the model constructs a feature vector by concatenating the learned representations of its left and right subtrees with features of the current join (e.g., join predicates, estimated selectivities, and row-width signals) in post-order; this embedding is then fed to a classifier that outputs *left-build* or *right-build*.

During training, labels are derived offline by comparing the observed output cardinalities of the two subtrees: if the left subtree’s cardinality is smaller, the label is *left-build*; otherwise, *right-build*. To maintain semantic consistency across recursive subplans, we assume that when evaluating a join node, all its descendant joins have already been assigned correct side selections. This assumption is enforced during training by restructuring query plans to satisfy it,

and during inference by traversing the plan bottom-up (see Fig. 4c), ensuring that lower-level joins are decided before higher-level ones. In this manner, the model adaptively determines build/probe selection throughout the query plan, improving memory efficiency and hash-table locality while preserving overall query semantics.

6 Multi-Modal Data Indexing and Hybrid Search

In ByteHouse’s production scenarios, there is a growing need to optimize the execution of multimodal search queries (e.g., text+vector retrieval in code assistant). While existing vector databases (e.g., Milvus-based solutions) are primarily designed for standalone similarity search, they lack tight integration with relational query execution. In contrast, ByteHouse builds upon a document-chunk abstraction (see Section 3.1.1) and extends standard SQL to support both vector and full-text search primitives within a single query. To efficiently execute such queries, ByteHouse introduces modality-aware optimizations, including runtime filtering, hybrid vector index designs, and multi-source result fusion strategies.

In the following, we introduce the design choices of vector indexes (in a hierarchical way) and hybrid data search in ByteHouse.

Multi-Layer Vector Indexes. Within a single table, we determine the suitable index structures based on the service types. The candidate vectors undergo a multi-layer structure tailored for various latency, freshness, and cost trade-offs. First, a coarse index layer performs partition-level pruning using product quantization (PQ) and BLAS-accelerated centroid distance, avoiding unnecessary index traversal. The rest layers of the index are determined based on the service types: (i) For latency-critical online services that demand high recall and millisecond-level responsiveness, ByteHouse adopts HNSW with scalar quantization (SQ), where vectors are pre-quantized into compact representations and organized in a navigable small-world graph. This allows bounded-depth traversal during query time, achieving fast search while maintaining recall. Under this setting, we can build the index asynchronously and decouple it from ingestion, ensuring minimal impact on write throughput. (ii) For near real-time services requiring high recall and rapid vector visibility (within seconds to sub-seconds), but with relaxed latency (e.g., 100ms-1s), ByteHouse adopts centroid-based partitioning during index construction, with each partition maintaining either full-precision vectors (IVFFlat), quantized vectors (IVFSQ), or PQ-based compression (IVFPQ). This enables fast ingestion-to-query cycles and moderate memory usage. (iii) For cost-sensitive services, ByteHouse adopts DiskANN that stores full-precision graph structures on SSDs and caches routing metadata in memory. Additionally, we use optimized I/O prefetching for common queries and beam search to maintain latency under relaxed constraints. For even more relaxed workloads (e.g., over long-tail vectors order than six months), ByteHouse supports DiskIVFSQ to store quantized, partitioned vectors on disk, significantly reducing both memory and compute costs, making it suitable for archival data with minimal freshness and latency demands.

Upon carefully constructed vector indexes, ByteHouse further extends its support to more complicated hybrid data search queries that blend semantic, textual, and structured constraints. Consider the hybrid query shown in Figure 5. The nested subquery retrieves

```

1 SELECT *
2 FROM (
3   SELECT a.document_id, a.chunk_id, a.chunk_content
4   FROM document_table
5   ORDER BY RANK_FUSION(
6     cosineDistance(a.embedding_col, [0.114, 0.257, ...]) ASC WEIGHT 1.0,
7     textSearch(a.text_col, 'ByteDance') DESC WEIGHT 2.0
8   ) DESC
9   LIMIT 100
10 ) AS a
11 JOIN (
12   SELECT *
13   FROM label_table
14   WHERE label_value = 'doc_image'
15 ) AS b
16 ON a.document_id = b.document_id
17 AND a.chunk_id = b.chunk_id;

```

Figure 5: Example Hybrid Data Search in ByteHouse.

the top- K chunks based on a fused ranking over both semantic similarity (via vector-based cosine distance) and keyword relevance (via lexical text search). These two modalities are integrated through the `RANK_FUSION` operator, which combines scores with configurable weights to balance semantic and lexical contributions. For the outer query, instead of issuing this vector-augmented query in isolation, we join its results with a filtered label table that selects only chunks annotated with a specific tag (e.g., “doc_image”). This join is structured to enable scalar-predicate-aware optimizations. ByteHouse executes such hybrid queries in three main steps.

(1) Cross-Table Runtime Filtering. First, ByteHouse conducts cross-table runtime filtering to prune the search space before initiating vector retrieval. The optimizer estimates the selectivity and optimal join order [46] (see Section 5). When the scalar-side table (e.g., `label_table`) is more selective, a runtime filter (e.g., Bloom filter, bitmap) is generated over the join keys and injected into the scan operator of the `document_table`. This early-stage pruning can both (1) improve the accuracy and (2) eliminate irrelevant rows. Otherwise, ByteHouse injects runtime filtering directly into operators like vector index scan, enabling coarse-grained pruning even within the vector retrieval phase.

(2) Fusion-based Hybrid Data Search. With the `RANK_FUSION` operator (used in Figure 5), ByteHouse brings vector and text search together with relation query processing. Specifically, `RANK_FUSION` is a specialized variant of the relational Union operator that offers two alternative fusion strategies. The first is a score-based fusion approach, which performs Min–Max normalization within each modality to align score scales, followed by weighted linear aggregation of the normalized scores. Alternatively, ByteHouse employs a rank-based Reciprocal Rank Fusion (RRF) algorithm [17], which aggregates results purely based on their relative positions in ranked lists. Given n ranked lists (e.g., from text and vector indexes), RRF assigns each document d a unified relevance score: $RRF(d) = \sum_{i=1}^n \frac{1}{k+r_i(d)}$, where $r_i(d)$ denotes the rank position of document d in the i -th modality-specific retrieval list, k is a smoothing constant (typically set to 60) that controls the decay of lower-ranked items, and n is the number of participating modalities. By operating on rank positions rather than absolute similarity scores, RRF achieves modality-agnostic and calibration-free fusion.

(3) Selective Post-Join Refinement. After the hybrid retriever produces the top- K candidates, ByteHouse performs a selective

post-join refinement to enforce structured predicates. Because runtime filtering (Step (1)) has already pruned most irrelevant rows, this join operates on a substantially reduced candidate set.

7 Performance Study

7.1 Standard Benchmark Comparison

We evaluate the analytical performance of ByteHouse operating against the latest StarRocks [59], Doris [7], and ClickHouse [52] using two representative benchmarks: TPC-DS (SF-1000) [63] and ClickBench [61]. All experiments are conducted on a dedicated cluster equipped with a 64-core AMD EPYC 9Y24 processor (128 hardware threads at 2.6 GHz) and 247 GB of main memory. Figure 6 summarizes the comparative results. On TPC-DS SF-1000 (Fig. 6a), ByteHouse consistently achieves lower end-to-end latency across all reported percentiles. The performance gaps widen at higher percentiles, such as P95 and P99, where multi-stage aggregations and joins dominate execution time and amplify engine-level bottlenecks. In these scenarios, ByteHouse achieves substantially lower tail latency than both StarRocks and Doris, indicating stable performance even under the most demanding queries. Fig. 6b presents per-query results on the ClickBench benchmark, which consists of 43 queries executed five times each, with the fastest run recorded. ByteHouse outperforms ClickHouse on approximately 32 queries and reduces the overall end-to-end latency by 25.4%. These performance gains are driven by ByteHouse’s high-parallelism execution engine, adaptive aggregation and grouping mechanisms, optimized Top-N processing paths, and an efficient memory allocator that reduces allocation overhead and fragmentation under high concurrency.

7.2 Effect of Incremental Processing

Figure 7 evaluates the efficiency of incremental processing mode (IPM) under representative inner join workloads from the TPC-H benchmark at scale factor 100. Figure 7a compares the CPU time of full recomputation and incremental computation across three typical join queries (Q12, Q14, Q19), where updates are applied to the `lineitem` table with an update ratio of 2.5%. The incremental approach consistently outperforms the full recomputation baseline, reducing CPU time by 28.4%–69.2%, which highlights its advantage for processing complex multi-join plans. Figure 7b further analyzes how incremental processing scales with different update ratios (2.5%, 5%, 10%) using join-only SQL queries without any filter predicates, with updates applied to either orders or `lineitem`. We measure the CPU time when applying updates to either the orders or `lineitem` table in query Q12. As the update ratio increases from 2.5% to 10%, the CPU time reduction grows from 19.9% up to 61.7%, indicating that the incremental strategy becomes increasingly beneficial as the magnitude of data updates rises. These results collectively demonstrate that incremental processing substantially improves computational efficiency, particularly in update-intensive analytical workloads.

7.3 Effect of CrossCache

We evaluate the performance of CrossCache on a dedicated ByteHouse cluster used for large-scale advertising analytics, including user conversion analysis across ad impressions, clicks, and exposure